

L03: Functions, Classes, and Modules

Dr. Todd Jones

t.r.jones@reading.ac.uk

Department of Computer Science

Continuing on from our understanding of Data Types, Operators, and Flow Control...

Lecture Objectives

- Understand and implement the concept of functional programming in Python.
 - **Define** and **call** functions.
 - **Assignments** and **scope**
 - **Anonymous** functions: `lambda`
 - **Higher-order** functions: `map` and `filter`
 - Understand and implement the concept of object-oriented programming (OOP) in Python.
 - **Building blocks:**
 - Class
 - Attribute
 - Method
 - Object
 - **Principles:**
 - Abstraction
 - Encapsulation
 - Inheritance
 - Polymorphism
 - Understand and implement the concept of modular programming in Python.
 - What is a module?
 - How to **define** a module.
 - How to **import** a module's contents.
-

Functions

- A **function** is a **set of statements** that take certain **inputs** (optional), performs specific **tasks**, and produces an **output** (optional).
- Many **intrinsic/built-in** functions are supplied, e.g.:

`pow(4,3)`

- The general objective of a function is to **reference** or **call** this block of code (*repeatedly*) instead of rewriting similar code multiple times.
 - **DRY: Don't Repeat Yourself**
 - Break long programs into smaller components.
 - Can be stored in modules to be shared and used by other program(mer)s.
- Analogy: **Mathematical (pure) functions**
 - $f(x)$ has **one** value.
 - For a given x , $f(x)$ is **always** the same.
 - Sometimes, different values of x can yield the same value for $f(x)$.
 - The pure function does not modify anything outside of itself.
 - No "side effects".
- **Design:**
 - Does your function do **one** thing?
 - Could someone else use your function without reading the rest of your notebook?
 - Is your function pure or does it rely on some external state?

Python Syntax:

- Just like `for`, `while`, `if`, and `else`, a **colon**, `:`, is required at the end of the **definition** statement.
- **Indentation** is required.
- **Parameters** are separated by commas.
 - *Parameters* are the *names* listed in the function *definition*.
 - *Arguments* are the actual *values* passed into the function when it is *called*.
- Type information for parameters and output is not required.
 - The `return` statement sends a value back to the place where the function was called.
 - Any object, or series of objects, can be returned.
 - Without a `return`, the function returns `None`.

Define a function that requires 2 arguments

```
def function_name(parameter1, parameter2, ):
```

```
    # Statement(s), e.g.:
```

```
    result = parameter1 + parameter2
```

```
    # Pass value back (optional)
```

```
    return result
```

Exercise: Write a function, named `function`, that accepts the appropriate input to calculate and output the result of:

$$f(x, y, z) = \frac{x^3 + y}{z}$$

```
In [ ]: # Writing the equivalent pure function in Python
# ???
```

In Python, functions are not restricted to this **pure** form. For instance, they can be designed to perform actions:

```
In [ ]: # This function has no arguments, and nothing is returned.
```

```
def print_message():
    print('I can print a message.')
```

```
In [ ]: # How does Python refer to this object type?
```

```
type(print_message)
```

```
In [ ]: # How is the object evaluated?
# Note: __main__ refers to the space of the top-level code,
#       the first user-specified Python module that starts running.
```

```
print_message
```

Describing Functions

We can request `help()` for built-in functions and most imported functions.

- And any other object.
- This is a wrapper around `pydoc` that will intelligently and deeply source system or developer-provided information.

```
In [ ]: help(pow)
```

```
In [ ]: help(print_message)
```

We can also **provide help** to other users by writing **docstrings** to describe our functions.

- Conventions vary by implementation: <https://peps.python.org/pep-0257/>
 - **PEP: Python Enhancement Protocols**
 - "A docstring is a string literal that occurs as the **first statement** in a module, function, class, or method definition. Such a docstring becomes the `__doc__` special attribute of that object."
 - List and describe the components contained within.
- Specially formatted docstrings can be **parsed** by other utilities to, for example, automatically create documentation for code bases.

- docutils: <https://docutils.sourceforge.io/>
- Sphinx: <https://www.sphinx-doc.org/en/master/>
- Automatically generated in some IDEs.

Example structure (*reStructuredText* format):

```
"""[Summary]

:param [ParamName]: [ParamDescription], defaults to [DefaultParamVal]
:type [ParamName]: [ParamType](, optional)
...
:raises [ErrorType]: [ErrorDescription]
...
:return: [ReturnDescription]
:rtype: [Return type]
"""
```

Other formats and guidelines: <https://realpython.com/documenting-python-code/>

```
In [ ]: # Incorporate a docstring in NumPy/SciPy format

def function(x, y, z):
    """
    Computes an invented example function with 3 parameters.

    Parameters
    -----
    x : int or float
        First parameter.
    y : int or float
        Second parameter.
    z : int or float
        Third parameter

    Returns
    -----
    (x**3 + y) / z : float
        Function result.

    """
    return (x**3 + y) / z
```

```
In [ ]: # Inspect result of help()

help(function)
```

```
In [ ]: # Inspect __doc__

function.__doc__
```

```
In [ ]: print(function.__doc__)
```

```
In [ ]: # Inspect other elements via tab completion
function.__
```

Automatic Docstring Generation

In Jupyter Notebooks

- ...not readily available.

- Fairly common: <https://marketplace.visualstudio.com/items?itemName=njpwerner.autodocstring>

Brave New World?

- Instruct your agent as to how you would like these formatted

Calling Functions and Arguments

- Functions are **called** by using their name and any associated arguments in a **statement** bracketed by parentheses `()`.

Let's call `print_message` :

```
In [ ]: print_message()
```

```
In [ ]: result = print_message()
        print(f'{result=}')
```

- Pass input values** by including them as **arguments** within the parentheses.
- Arguments might be:
 - positional**: need to be provided in a certain order
 - keyword**: are associated with a specific variable name that is used within the function (provide in any order).

Let's test this with `function` :

```
In [ ]: # Provide positional arguments
        function(3, 2, 1)
```

```
In [ ]: # Provide keyword arguments (any order)
        function(z=1, y=2, x=3)
```

```
In [ ]: # Provide both positional and keyword arguments
        function(z=1, 2, x=3)

# This call will fail because positional arguments must be supplied
# before keyword arguments.
```

```
In [ ]: # Provide both positional and keyword arguments
        function(3, z=1, y=2)
```

```
In [ ]: # Provide arguments as named variables
        first = 3
        second = 2
        third = 1

        function(first, second, third)
```

Optional arguments are **defined** as keyword arguments: `function(keyword=value):`.

- The **values** are often taken to be the **default** parameters of a function, perhaps to be used if the caller forgets to supply a value.

Commonly seen in data science libraries, e.g.:

```
def sklearn.tree.DecisionTreeClassifier(
    *, criterion='gini', splitter='best', max_depth=None, min_samples_split=2,
    min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None,
    random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0,
    class_weight=None, ccp_alpha=0.0):
```

```
In [ ]: # Positional arguments MUST be supplied to operate the function:
function(3, 2)
```

```
In [ ]: # Define the function with z as a keyword argument, defaulting to 1
def kwfunction(x, y, z=1):
    return (x**3 + y) / z
```

```
In [ ]: # Keyword arguments are optional:
kwfunction(3, 2)
```

```
In [ ]: # Keyword arguments can be used as positional arguments,
# consistent with their definition
kwfunction(3, 2, 1)
```

```
In [ ]: # Keyword arguments can be specified by their keyword
kwfunction(3, 2, z=6)
```

Parameter structure is not just syntax; it is part of a function's "contract" with its caller.

Arbitrary sets of arguments can be handled by **packing** and **unpacking** of iterables and dictionaries.

- `*args` : positional arguments
- `**kwargs` : keyword arguments

`*` unpacks or "explodes" an iterable object.

`**` unpacks or "explodes" a dictionary (i.e., key:value) object.

Let's arrange our arguments to a new function as an iterable list:

```
In [ ]: # Unnamed, positional arguments
l_args = ['first', 'second', 'third']

def order(one, two, three):
    print(one, two, three)

# Evaluate l_args as a set of positional arguments
order(*l_args)
```

Let's expand the function to accept keyword arguments:

```
In [ ]: # Named, keyword arguments
d_args = {'four': 4,
          'five': 'five',
          'six': 6.0}

def order(one, two, three, four=None, five=None, six=None):
```

```
print(one, two, three, four, five, six)

order(*l_args, **d_args)
```

In general, the expected order is structured like:

```
def function(arg1, ..., argN, *args, kwarg1, ..., kwargN, **kwargs):
```

Note: In the function definition `*` and `**` are instructing the function to *pack* any arguments that might be provided into these parameter names.

Let's supply a combination of explicit and expandable arguments:

```
In [ ]: parg1 = 'parg1'
l_args = ['second', 'third']
d_args = {'four': 4,
          'five': 'five'}

order(*l_args, parg1, **d_args )
# Note the use of default values for unused kwargs
```

```
In [ ]: order(*l_args, parg1, six='sixth', **d_args)
```

-
- Similarly, you can define functions to **expect** an unknown number of positional or keyword arguments.
 - When defined with unknown positional arguments, any keyword arguments must be explicitly provided to the function.

```
In [ ]: def arg_test(arg1, *argvals, kwarg1=None, **kwargpairs):
print("First argument:", arg1)
for anc in argvals:
    print("Another argument from *argvals:", anc)

print('\nSpecified keyword:', kwarg1)
for key, value in kwargpairs.items():
    print(f"Found keyword arg: '{key}': {value}")

kws = {'kwarg2': 22,
       'kwarg3': 33,
       'kwarg4': 44,
       'kwarg5': 55}
arg_test('name', 100, 200, 300, 'positional?', **kws)
```

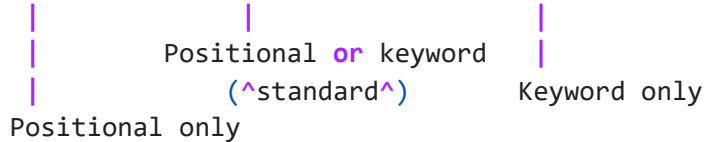
Positional-Only and Keyword-Only Arguments

The default for supplying function arguments is that they may be invoked with their parameter name. That is, they may be supplied positionally or by keyword, as we saw above.

To enhance readability and performance, we can restrict and **specify** how arguments should be provided.

In general, the expected order is structured like:

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
    -----
```



The *optional* `/` and `*` entries indicate which elements are positional or keyword.

- Parameters before the `/` are positional-**only**.
- Parameters following the `/` may be **either** positional-or-keyword or keyword-only.
- Parameters after the `*` are keyword-**only**.
- These indicators can be used:
 - not at all
 - one at a time
 - together.

<https://docs.python.org/3/tutorial/controlflow.html#keyword-arguments>

Parameters are passed to functions depending on their **mutability**:

- Immutable**s, such as integers and strings are passed by **value**.
- Mutable**s, such as lists, are passed by **reference**.

```
In [ ]: # Pass by value: immutable (e.g., string)

def duplicate(in_string):
    in_string = in_string * 2
    return in_string

x = '! Some message !'
print(f'Input Before: \n\t{x=}')

y = duplicate(x)

print(f'Input After: \n\t{x=}')
print(f'Output: \n\t{y=}')

# The original input variable is unchanged.
```

```
In [ ]: # Pass by reference: mutable (e.g., list)

def swap(a):
    a[0], a[-1] = a[-1], a[0]
    return a

x = [0, 1, 2, 3, 4, 5, 6]
print(f'Input Before: \n\t{x=}')

y = swap(x)

print(f'Input After: \n\t{x=}')
print(f'Output: \n\t{y=}')

# The original input variable is CHANGED.
```

```
In [ ]: # Consider...

new_string = 'ABCDEFGHIIJKLMNOP'
```

```
print(new_string[0], new_string[-1])
```

Assignments and Scope

Where is a value accessible?

- Variables in Python need to be **assigned** before they can be used.
 - The location of the assignment of a variable determines its **scope**, i.e. where it can be accessed.
-

Scope in Python is determined by the **LEGB** rule:

- **L**ocal - names inside a function.
- **E**nclosing - names in enclosing functions (nested functions).
- **G**lobal - names in the current module.
- **B**uilt-in - names in the built-in namespace.

As such, Python determines the assigned object by looking for variable names in that order.

- Assignments made at the top level are **global** and accessible from **anywhere**:

```
In [ ]: # Define value at the top level (global)
a_number = 88

def add_number(inval):

    # a_number is in scope here
    return inval + a_number

add_number(2)
```

-
- Variables assigned within functions are **local** to that function definition:

```
In [ ]: def dummy():
    dval = 10
    print(f'Inside function: {dval}.')

dummy()

print(f'Outside function: {dval}.')
```

-
- In contrast to functions, assignments made within `for` or `while` loops are "in scope" for the level at which the **loop** is defined:

```
In [ ]: for inc in [1, 2, 3]:
    aaa = 'defined' # still in the global space

print(aaa)
```

-
- References to objects will first search the **local** scope, then **global**.

- i.e. if a variable is assigned in the local scope, the *local version* will be found.

```
In [ ]: dval = 5

def dummy():
    dval = 10
    print(f'Inside function: {dval}.')

dummy()

print(f'Outside function: {dval}.')
```

Trying to reference the global variable in the assignment will not bypass this, since the interpreter recognises that the variable *will be* assigned in the local scope:

```
In [ ]: dval = 5

def dummy():
    dval = dval + 10
    print(f'Inside function: {dval}.')

dummy()

print(f'Outside function: {dval}.')
```

-
- A variable must be formally recognised as a global variable within the local scope (with the `global` keyword) to modify the global value.
 - **BE CAREFUL WITH THIS!**
 - This permits functions to modify global variables.

```
In [ ]: dval = 5

print(f'Outside function: {dval}.')

def dummy():
    global dval
    dval = dval + 10
    print(f'Inside function: {dval}.')

dummy()

print(f'Outside function: {dval}.')
```

```
In [ ]: # A list of globally-available variables is accessible with:
globals() # This will display a lot of information.

# [output removed for presentation, incredibly long]
```

Anonymous Functions: `lambda`

- In Python, functions are **first-class**:
 - They can be treated as **values**, having names, being assigned to variables.
- In Python, functions can be **higher-order**:

- They can be **passed** to other functions.
- They can be **returned** from other functions.

As an example of this, we can look at **anonymous functions**. In Python, these are generated by the `lambda` keyword:

lambda arguments : expression

- This can take any number of arguments but can have **only one** expression.
- Often used as arguments to higher-order functions.
- Ideal for single-use situations or temporary use.

```
In [ ]: # Define a lambda function
        lambda arg1, arg2 : (arg1 / arg2, arg1 + arg2)
```

-
- Although no name is assigned to the function when it is defined, you can **assign the function to a variable** just like you would with any other value:

```
In [ ]: # Define, store, and call a lambda function
        lfunc = lambda arg1, arg2: (arg1 / arg2, arg1 + arg2)
        lfunc(5, 10)
```

```
In [ ]: # As a function:
        def lfunc(arg1, arg2):
            div = arg1 / arg2
            add = arg1 + arg2
            return (div, add)
        lfunc(5,10)
```

-
- To use `lambda` with a conditional statement, we can use the **ternary** operator form:
 - https://book.pythontips.com/en/latest/ternary_operators.html

```
In [ ]: is_odd = lambda x: f'Nope! {x} is not odd.' if x % 2 == 0 else 'Yeah!'
        is_odd(3)
```

Higher-order Functions:

- **Higher-order** functions are functions that **accept** other functions as arguments and operate on them, or that **return** functions as results.
- In programming, higher-order functions can be used to write **customisable functions**, whose operations depend on their arguments. They can also be used to build functions.
 - We will come back to these concepts later in the course.

`map()`

The intrinsic `map()` function applies a given function to **each element** of a collection and returns an **iterator**.

- **Readability**: makes it clear that you are applying the same function to each element.
 - **Correctness**: combine existing built-in functions
 - **Parallelisable**: can be processed in **parallel** on multiple cores/threads or different computers.
 - Requires use in standard library or third party tool implementations.
 - See:
<https://docs.python.org/3.13/library/multiprocessing.html#multiprocessing.pool.multiprocessing.Pool>
-

Usage:

`map(function, collection_inputs, *iterables)`

```
In [ ]: # Map a cubing function onto a list of numbers:
values = [10, 20, 30, 40, 50]
cubed = map(lambda x: x**3, values)
cubed
```

```
In [ ]: # Step through an iterator with next()
next(cubed)
```

```
In [ ]: # Use list() to get all values in one object:
cubed = map(lambda x: x**3, values)
list(cubed)
```

```
In [ ]: # Use a built-in function to calculate the sums of 3 lists of numbers

num0 = [1, 2, 3, 4, 5]
num1 = [11, 12, 13, 14, 15]
num2 = [21, 22, 23, 24, 25]

list(map(sum, [num0, num1, num2]))
```

```
In [ ]: # Use built-in functions as the inputs

for i in range(1, 5):
    print(list(map(lambda x: x(range(i)), [sum, min, max])))
```

filter()

The `filter()` operation applies a condition to each member of a collection and **returns an iterator** containing only members for which the condition was true.

- The function should return only `True` or `False`.

`filter(function, collection_inputs)`

```
In [ ]: # From a dictionary of scores, create a dictionary of those
# reaching First Class
scores = {'Jehad': 75,
          'Andrew': 72,
          'Rand': 82,
          'Imole': 60,
```

```
'Ronit': 51,  
'Hamza': 77,  
'Stefan': 33,  
'Kat': 95,  
'Atta': 15}
```

```
dict(filter(lambda pair: pair[1] >= 70, scores.items()))
```

Object-Oriented Programming

Object-Oriented programming (OOP) is a computer programming model that organizes software design around data, or objects.

The structure, or building blocks, of object-oriented programming include:

- **Classes:** user-defined data types that act as the blueprint for individual objects, attributes and methods.
- **Attributes:** defined within the class template and represent the state of an object. Objects will have data stored in the attribute fields.
- **Methods:** functions that are defined inside a class that describe the behaviors of an object.
- **Objects:** instances of a class created with specifically defined data.

```
class className:  
    attribute = ...  
  
    def method():  
        ...
```

```
instance_object = className()
```

This is the core of Python!

Abstraction

- Concept to handle complexity by **hiding unnecessary implementation details**.
- This enables programmers to implement more complex logic on top of the provided abstraction without understanding or even thinking about all the hidden complexity.

Encapsulation

- Data and the functions that work on them are **bound together** in objects.
- When a class is defined, all attributes and methods are **public by default**.
 - any attributes of a class instance can be modified outside of the class.
- We might want to hide variables from view outside of the object to provide some **security**.
- To declare attributes/methods as **private**, use *double underscore* (`__`) in the prefix.

Defining a class

- **self** : To allow our methods to work with distinct instances of objects, **methods take an extra first argument**, by convention, called **self**.
 - This is a **reference to the instance** of the object that the method was called on.
- **__init__** : the object **constructor**; a special method that the program calls upon an object's creation, to **initialise** data members to the object (instance attributes).
 - A "**dunder**" method.
- **__str__** : return a **printable, readable** string representation of an object.
 - To be read easily by others.
- **__repr__** : a **representation** of an object.
 - `<type(self).__module__ type(self).__qualname__ at hex(id(self))>`
 - Return a string representation of an object that can possibly be evaluated with `eval()`.
 - Printed when no `__str__` method is supplied.
 - Displayed when writing an objects name on an interactive console.
- **Class variables**: defined inside of a class, these assignments will be made when the class is **defined**, and are shared attributes between **all instances** of the class.
- **Instance variables**: data which is **unique** to every object.
- **Private variables**: "Private" instance variables that cannot be accessed except from inside an object **do not exist** in Python.
 - However, there is a **convention** that is followed by most Python code: a name **prefixed with an underscore** (e.g., `_spam`) should be treated as a non-public part of the API (whether it is a function, a method, or a data member).
 - Under **name mangling**, any identifier of the form `__spam` (at least two leading underscores, at most one trailing underscore) is *textually replaced* with `_classname__spam`.
 - Only performed **upon class definition**.

Let's create a **class** to define and modify student properties.

```
In [ ]: # Define a class with one class variable
class Student:
    name = 'Antonio'

# Create and inspect an instance of this class
s = Student()
s.name
```

```
In [ ]: # Modify the class variable
Student.name = 'Shirley'

# Inspect the instance
s.name
```

```
In [ ]: # repr
repr(s)
```

```
In [ ]: # str
print(s)
```

```
In [ ]: # Create a new instance of the class and inspect the associated name
s2 = Student()
s2.name
```

```
In [ ]: # Define a class with instance variables
class Student:
    """
    Descriptive formal docstring...

    Contains...methods...

    """

    # Class attribute
    count = 0

    # Constructor method
    def __init__(self, name='Default', mark=0):
        # Initialise instance attributes
        self.name = name
        self.mark = mark

        # A "private" attribute
        self.__secret = 'HUSH'

        # Access the class attribute
        Student.count += 1
        self.sid = Student.count

        # Initialise empty dictionary
        self.modules = {}

    # Method to print information about class
    def info(self):
        print(f'{self.name}, {self.sid}: {self.mark}, {self.__secret}')

    # String representation
    def __str__(self):
        return self.name

    # Object representation
    def __repr__(self):
        return f'{type(self).__qualname__}("{self.name}",{self.mark})'

    # An instance method
    def add_submark(self, module, mark):
        if module in self.modules:
            self.modules[module].append(mark)
        else:
            self.modules[module] = [mark]

        self.__update_mark()

    # A private instance method
    def __update_mark(self):
        total = 0
        entries = 0
        for key, value in self.modules.items():
```

```

        for v in value:
            total += v
            entries += 1

        self.mark = total / entries

    # Polymorphism: operator overloading
    def __add__(self, other):
        return f'{self.name} and {other.name}'

    # Polymorphism: allow subclasses to define behaviour
    def classification(self):
        raise NotImplementedError("Subclass must implement abstract method.")

```

```
In [ ]: # Inspect the class
dir(Student)
```

```
In [ ]: # Create an instance
s1 = Student()
s1.info()
```

```
In [ ]: # Instantiate and inspect additional students
s2 = Student('Zaki', 72)
s3 = Student('Mehdi', 50)
s4 = Student('Akash', 88)

s2.info()
s3.info()
s4.info()
```

```
In [ ]: # Add marks for different modules
s4.add_submark('Python', 66)
s4.add_submark('Python', 84)
s4.add_submark('Java', 77)
```

```
In [ ]: # See that the overall mark has been updated
s4.info()
```

```
In [ ]: # Inspect modules for a Student instance
s4.modules
```

```
In [ ]: # Inspect the "private" method of an instance
s2._Student__update_mark
```

```
In [ ]: # Inspect the "private" attribute of an instance
s2._Student__secret
```

```
In [ ]: # Set an attribute
s2.__secret = '???'
s2.__secret
```

```
In [ ]: s2._Student__secret
```

```
In [ ]: dir(s2)
```

```
In [ ]: # Similarly, but with a focus on the attributes of an instance and their values:
vars(s2)
```

```
In [ ]: s2.info()
```

```
In [ ]: # Inspect the shared class variables
print(s1.count)
print(s4.count)
```

Inheritance

- A mechanism to **derive a class from another class** to create a **hierarchy of classes** that share a set of attributes and methods.
- It is a way of creating a **new class** while using details of an existing class without modifying it.
- The original class is a **base class** (or **parent class**).
- The newly formed class is a **derived class** (or **child class**).
 - We can expand behaviour in the child class. The "**extensibility**" concept.
 - Define with parent class name input.
 - We can initialise with `super().__init__()` to construct the parent class first.
 - This is default behaviour and does not need to be supplied explicitly.
 - Use the `super()` function to call methods from the parent class.

```
In [ ]: # A child class which inherits attributes name, mark and functions
# from the parent class
class UGStudent(Student):

    def __init__(self, name, mark):
        super().__init__(name, mark)    # run the constructor of the parent class

us1 = UGStudent('Adam', 75)
```

```
In [ ]: # Inspect the new UGStudent instance
us1.info()
```

```
In [ ]: # Check the class counts
s4.count, us1.count
```

```
In [ ]: # Test class membership - Is this instance a member of the UGStudent class?
isinstance(us1, UGStudent)
```

```
In [ ]: # Test class membership (parent)
isinstance(us1, Student)
```

```
In [ ]: # A child class which inherits attributes name, mark and functions
# from the parent class and builds on them
class UGStudent(Student):

    # Constructor
    def __init__(self, name, mark, contact): # now positional, required parameters
        super().__init__(name, mark)    # run the constructor of the parent class
        self.emergency_contact = contact

    # Method to print information about class
    def info(self):
        print(f'{self.name}, {self.sid}: {self.mark}. Contact: {self.emergency_contact}')

us1 = UGStudent('Adam', 75, 'Mother')
us1.info()
```

```
In [ ]: # A variation on previous:
# .info() method relies on parent functionality, then adds functionality

# A child class which inherits attributes name, mark and functions
# from the parent class and builds on them
class UGStudent(Student):

    # Constructor
    def __init__(self, name, mark, contact):
        super().__init__(name, mark) # run the constructor of the parent class
        self.emergency_contact = contact

    # Method to print information about class
    def info(self):
        super().info()
        print(f'Contact: {self.emergency_contact}')

us1 = UGStudent('Adam', 75, 'Mother')
us1.info()
```

Polymorphism

- This is a Greek-based word constructed from "poly", meaning many, and "morph", meaning forms. So Polymorphism means having **many forms**.
 - In OOP in Python, we can see instances of this through:
 - Operator overloading: redefine an operator for a `class`
- ```
Polymorphism: operator overloading
def __add__(self, other):
 return f'{self.name} and {other.name}'
```
- Child class modifies behaviour of a parent method:
- ```
def classification(self):
    if self.mark >= 70:
        ...
```
- ^^ Implement in practical. ^^

```
In [ ]: # Operator Overloading
s1 + s2
```

- Related but distinct: **duck typing**.
 - Use of 2 **different** classes with the **same method names**.
 - We are able to use instances of classes without specifying that a variable must be an object of that **specific class**.
 - This means that we can define classes whose objects behave much like objects of **another class, without inheritance**.

```
In [ ]: # Duck typing in class methods

class fruit:

    def __init__(self, kind):
```

```

        self.kind = kind

    def info(self):
        print(f"I am a fruit: {self.kind}.")

    def calculate(self, values):
        return(sum(values))

class vegetable:

    def __init__(self, kind):
        self.kind = kind

    def info(self):
        print(f"I am a vegetable: {self.kind}.")

    def calculate(self, values):
        return(max(values))

# Implement methods of the two classes in the same manner
for x in [fruit('strawberry'), vegetable('spinach')]:
    x.info()
    print(f'Calculation result: {x.calculate([23, 45, 67])}')
    print()

```

Summary and Independent Study Resources

Functions

- A set of statements that take certain inputs (optional), perform specific tasks and produces an output (optional).
 - **Immutable**s such as integers and strings are **passed by value**.
 - **Mutables** such as lists are **passed by reference**.
 - **Global variables** are accessible from anywhere.
 - **Local variables** assigned within functions are local to that function definition.
- A **lambda** function is a small anonymous function, a quick way to define a function.
- **map()** applies a given function to each item of an iterable (list, tuple etc.) and returns an iterator.
- **filter()** extracts elements from an iterable (list, tuple etc.) for which a function returns True.

Object-Oriented Programming (OOP)

- **Classes** are user-defined data types that act as the blueprint for individual objects, attributes and methods.
- **Attributes** are defined in the class template and represent the state of an object. Objects will have data stored in the attributes field.
- **Methods** are functions that are defined inside a class that describe the behaviors of an object.
- **Objects** are instances of a class created with specifically defined data.
- **Abstraction** - hiding the internal details or implementations of a function and showing its functionalities only.
- **Encapsulation** - hides the data from the access of outsiders.
- **Inheritance** - child class inherits the attributes and methods of the parent class. A child class can have its own properties and methods.

- **Polymorphism:**
 - Operator overloading
 - Behaviour modification
 - Duck typing

Online Tutorials:

- Functions: <https://www.programiz.com/python-programming/function>
- `lambda` : <https://www.programiz.com/python-programming/anonymous-function>
- `map` <https://www.programiz.com/python-programming/methods/built-in/map>
- `filter` <https://www.programiz.com/python-programming/methods/built-in/filter>
- Starred expressions: <https://www.geeksforgeeks.org/starred-expression-in-python/>
- Instance Variables: <https://pynative.com/python-instance-variables/>
- Class Variables: <https://pynative.com/python-class-variables/>
- Class methods: <https://www.programiz.com/python-programming/methods/built-in/classmethod>
- OOP concepts: <https://www.edureka.co/blog/object-oriented-programming-python>

References:

- Lutz, M. (2025). Learning Python: Powerful Object-Oriented Programming
 - **Parts IV and VI**
- McKinney (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython.
 - **Chapter 3**

Extension Resources:

On the Blackboard Solution page, you will see:

- Discussions and studies on functional and OOP constructs
- Case Study on "Python Module Shadowing"

Homework: Copies and Modules

Refer to the materials under **Week 03 on Blackboard**. Work through and read about:

- issues in **copying** variables
- the use and creation of **module** files in Python

In This Week's Practical

Practice using functions, classes, and modules.

Next Week

Error Handling, Testing, I/O, and Data Files

